



Bachelor Thesis

Physical Validation of Temporal Explainability in Quantized
Spiking Neural Networks for Gravitational Wave Glitch
Classification: A Hardware-in-the-Loop Approach

MUCAJ DEVA
Artificial Intelligence

Academic Year 2025/2026

Bachelor in Artificial Intelligence



Mucaj Deva

Physical Validation of Temporal Explainability in Quantized Spiking Neural Networks for Gravitational Wave Glitch Classification: A Hardware-in-the-Loop Approach

Supervisors: Claudio Zandron, Department of informatics,
systems and communication, UniMiB
Second Supervisor, Affiliation

Abstract

Spiking Neural Networks (SNNs) promise a low-power alternative to conventional Deep Neural Networks for deploying explainable classifiers on memory-constrained edge devices. Applying this promise to a scientific domain, this thesis investigates whether a severely quantized SNN can retain its ability to identify the physical features of gravitational-wave detector glitches from the Gravity Spy dataset, and whether the temporal explanations produced by the software model are corroborated by the physical behaviour of the hardware that finally runs it.

An SNN classifier is trained on spectrogram representations of a subset of Gravity Spy glitch classes using surrogate-gradient backpropagation, and then compressed through Quantization-Aware Training (QAT) applied to both synaptic weights and membrane potentials. Temporal Spike Activation Maps are extracted from the baseline and quantized models to identify, at the timestep level, when and on which time-frequency regions each network bases its decision, following a Time-to-First-Spike (TTFS) coding scheme. The quantized model is then ported to a C/C++ inference loop on a microcontroller (ESP32/Teensy), and a Hardware-in-the-Loop (HIL) protocol is used to physically measure the inference latency: an output GPIO pin is toggled on the first output spike and captured with an oscilloscope/logic analyzer, isolating the physical TTFS from operating-system and software-simulation artefacts.

This work contributes (i) a hardware-friendly quantization pipeline for an SNN trained on astrophysical spectrogram data, (ii) a temporal, per-timestep explainability method for comparing baseline and quantized models, and (iii) a physical validation protocol that correlates software-predicted spike timing with oscilloscope-measured GPIO latency. The resulting comparison between theoretical and physically measured temporal explanations is intended to clarify whether extreme quantization preserves not only classification accuracy but also the physical faithfulness of temporal explainability maps, informing the trade-offs relevant to future neuromorphic deployments in gravitational-wave observatories.

Keywords

Spiking Neural Networks, Explainable AI, Quantization-Aware Training, Time-to-First-Spike, Hardware-in-the-Loop, Gravity Spy, Neuromorphic Computing, Edge AI

Contents

1	Introduction	3
1.1	Background and Context	3
1.2	Problem Statement	4
1.3	Research Objectives and Contributions	4
1.4	Thesis Structure	5
2	State of the Art and Theoretical Framework	6
2.1	Spiking Neural Networks and Edge AI	6
2.2	Temporal Explainability in SNNs	7
2.3	Hardware-in-the-Loop Validation Profiling	8
2.4	The Gravity Spy Dataset	8
3	Software Methodology: SNN Design and Explainability	9
3.1	Dataset Preparation and Preprocessing	9
3.2	Baseline Model Architecture and Training	10
3.3	Quantization-Aware Training (QAT) Strategy	10
3.4	Extraction of Temporal Spike Activation Maps	11
4	Hardware Implementation: Physical Validation Setup	12
4.1	Model Porting and Embedded Deployment	12
4.2	Hardware-in-the-Loop (HIL) Architecture	13
4.3	Logic Analyzer Measurement Protocol	13
5	Results and Discussion	15
5.1	Impact of Severe Quantization on Accuracy	15
5.2	Software Explainability Analysis	16
5.3	Physical Validation of Temporal Maps	16
5.4	Trade-offs and Neuromorphic Feasibility	17
6	Conclusions and Future Work	19
6.1	Summary of Key Findings	19
6.2	Limitations of the Study	19
6.3	Future Directions	20
	References	22
A	Hardware Schematics and Pinouts	24
B	Embedded Inference Code	25

Chapter 1

Introduction

1.1 Background and Context

Machine learning has become a fundamental tool in astrophysics, notably in the analysis of data from ground-based gravitational-wave detectors, like the Advanced LIGO (Laser Interferometer Gravitational Wave Observatory) and the Advanced Virgo.

A main application in this field is the detection and analysis of “glitches” – short, complex noise transients that disrupt the identification of true wave signals. Researchers currently successfully implement Deep Convolutional Neural Networks (CNNs) trained on the citizen-science Gravity Spy dataset, a public project that combines human intuition and pattern recognition with advanced machine learning algorithms to characterize and classify the diverse noise transients based on 2D time-frequency representations.

While standard deep learning models provide high classification accuracy, they face two significant limitations in this domain: a lack of transparency and massive computational requirements.

First, scientific applications as a whole demand a profound and rigorous understanding of the underlying physical mechanisms behind observed data. Gravitational-wave observatories in particular record tens of thousands of auxiliary channels monitoring instrumental and environmental conditions, enlarging significantly the volume of data to be processed. This vast amount of data, paired with traditional neural networks’ opaque “black boxes”, explains the need for more transparent and efficient models. In this context, Explainable AI (XAI) has emerged as a pivotal solution, managing to trace predictions back to physical root causes, ensuring that automated discoveries are reliable and free from instrumental biases.

Second, traditional Deep Neural Networks require significant power and computational resources due to multiply-and-accumulate (MAC) operations. To overcome this energetic bottleneck, neuromorphic computing and Spiking Neural Networks (SNNs) offer an efficient alternative. Inspired by biological systems, SNNs process information through sparse, asynchronous binary spikes, utilizing low-cost accumulate (AC) operations, which are ideal for low-power neuromorphic hardware platforms.

Despite their theoretical energy advantage, deploying SNNs on memory-constrained edge devices presents a critical architectural challenge, generally referred to as the *memory wall*. SNNs are indeed memory-intensive, since they must store both synaptic weights and the temporal states of each Leaky Integrate-and-Fire (LIF) neuron, known as membrane potentials. As synaptic weights are quantized to lower precisions to preserve space

constraints, membrane potentials can consume up to 98% of the total memory footprint during batch processing. Therefore, successfully deploying high-performance SNNs on resource-constrained systems requires hardware-friendly compression techniques that can quantize both weights and membrane potentials without collapsing the network’s predictive accuracy and, as this thesis argues, without silently collapsing the reliability of its explanations.

1.2 Problem Statement

The deployment of Spiking Neural Networks (SNNs) on memory-constrained edge devices necessitates severe quantization of both synaptic weights and membrane potentials. While converting these parameters from high-precision floating-point representations to ultra-low bit-width integers (e.g., 2-bit or 4-bit) drastically reduces memory footprints and eliminates the need for power-intensive multipliers, it simultaneously risks altering the spatiotemporal dynamics that govern the network’s decision-making process.

Currently, the evaluation of quantization techniques for SNNs is predominantly conducted through software simulations or on specialized Field-Programmable Gate Array (FPGA) accelerators [7]. Consequently, a significant research gap exists regarding the physical interpretability of these highly compressed models when deployed on commodity microcontrollers. Although explainability is critical in scientific applications such as astrophysics, existing literature focuses heavily on optimizing accuracy and energy efficiency rather than understanding how quantization distorts internal feature representations. Furthermore, the provided literature does not cover the physical correlation of theoretical explainability maps with real-time execution metrics. Specifically, there is a lack of methodology to prove whether the theoretical temporal spike attributions generated in software accurately correspond to the physical, real-time latency of the neuromorphic hardware.

This thesis aims to bridge the gap between theoretical software-based Explainable AI (XAI) and the physical execution of highly compressed SNNs on resource-constrained hardware. By focusing on the complex time-frequency morphologies of the Gravity Spy dataset, this research systematically investigates how extreme quantization impacts the physical interpretability of neuromorphic models.

1.3 Research Objectives and Contributions

The primary objectives and contributions of this work are three-fold:

1. **Hardware-Friendly Quantization for Astrophysical Data:** To implement and train an SNN classifier on a subset of the Gravity Spy dataset, applying extreme, integer-only quantization to both synaptic weights and membrane potentials (e.g., 2-bit or 4-bit precision) to simulate the strict memory constraints of neuromorphic chips.
2. **Extraction of Temporal Explainability Maps:** To apply and evaluate temporal XAI algorithms – specifically Spike-CAM and Temporal Spike Attribution – comparing the high-precision baseline model against the quantized model, in order to determine at which timestep t each network identifies the physical features of a glitch, and whether that timestep is consistent between the two models.

3. **Physical Validation of Temporal XAI Metrics:** To deploy the heavily quantized SNN onto a microcontroller (ESP32/Teensy) and physically validate the theoretical temporal XAI predictions. This involves toggling a GPIO pin on the first output spike and reading it with an oscilloscope/logic analyzer, correlating the real-time execution latency (physical Time-to-First-Spike) with the theoretical explainability maps generated in software.

Taken together, these three objectives make it possible to answer this thesis' central question: *does a severely quantized SNN still classify gravitational-wave glitches based on genuine physical features, and does the timing of its physical response on real hardware match what its software-generated explanation predicted?*

1.4 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 establishes the theoretical framework, reviewing the state of the art in spiking neural networks (SNNs) for edge AI, temporal explainability paradigms, Hardware-in-the-Loop profiling standards, and the astrophysical significance of the Gravity Spy dataset. Chapter 3 details the software methodology, encompassing dataset preprocessing, the baseline SNN architecture and training procedure, the Quantization-Aware Training (QAT) strategy, and the programmatic extraction of Temporal Spike Activation Maps used to identify decision-making at specific timesteps (t). Chapter 4 describes the hardware implementation, outlining the translation of quantized weights for embedded deployment, the Hardware-in-the-Loop (HIL) architecture, and the logic-analyzer measurement protocol used to capture physical inference latency. Chapter 5 presents the experimental results, comparing the classification performance of the baseline and quantized models, contrasting their software explainability maps, and rigorously comparing theoretical software explainability against physical oscilloscope traces, while evaluating the trade-offs between memory compression and XAI fidelity on constrained devices. Finally, Chapter 6 summarizes the primary findings, acknowledges the architectural limitations of the experimental setup, and proposes avenues for future neuromorphic applications.

Chapter 2

State of the Art and Theoretical Framework

This chapter surveys the four bodies of knowledge that this thesis builds upon. Section 2.1 reviews Spiking Neural Networks, the Time-to-First-Spike coding paradigm, and the memory constraints that motivate quantization on edge devices. Section 2.2 discusses temporal explainability techniques for SNNs. Section 2.3 introduces Hardware-in-the-Loop profiling and the industry standards that justify GPIO-based latency measurement. Section 2.4 closes the chapter with the astrophysical context of the Gravity Spy dataset.

2.1 Spiking Neural Networks and Edge AI

Spiking Neural Networks are often described as the third generation of neural network models [10]. Unlike conventional Artificial Neural Networks (ANNs), which propagate real-valued activations synchronously through dense multiply-and-accumulate operations, SNNs communicate through sparse, binary events (spikes) that are integrated over discrete timesteps by biologically inspired neuron models. The most widely used abstraction is the Leaky Integrate-and-Fire (LIF) neuron, whose membrane potential $V(t)$ accumulates weighted input spikes and leaks over time until it crosses a firing threshold V_{th} , at which point the neuron emits a spike and its potential is reset. Because information is encoded in the presence and timing of these events rather than in continuous activations, SNN inference can, in principle, be implemented with low-cost accumulate-only (AC) arithmetic, giving neuromorphic hardware platforms an intrinsic energy advantage over MAC-based accelerators [10, 2].

Training SNNs is complicated by the non-differentiability of the spike (Heaviside) function. This thesis follows the now-standard *surrogate gradient* approach [9], in which the discontinuous derivative of the spiking non-linearity is replaced, only during the backward pass, with a smooth surrogate (e.g., a fast sigmoid or arctangent), allowing gradient-based optimization frameworks such as PyTorch to train SNNs end-to-end through Backpropagation Through Time (BPTT). The `snnTorch` library [3] implements this approach and is used throughout this work to define, train, and simulate the LIF-based network.

Among the possible information-coding schemes for SNNs (rate coding, temporal/latency coding, population coding), this thesis adopts *Time-to-First-Spike* (TTFS) coding [8]. In a TTFS scheme, information is encoded in the latency of the first spike emitted by a

neuron after stimulus onset: a strongly excited neuron fires early, a weakly excited one fires late or not at all. TTFS coding is attractive for two reasons that are central to this thesis. First, it is inherently sparse – at most one spike per neuron is needed to convey a decision, minimizing the number of switching events that must be measured on hardware. Second, and more importantly for this work, it provides a natural link between a *software* quantity (the timestep t at which a neuron first fires in simulation) and a *physical* quantity (the wall-clock delay between stimulus presentation and the corresponding GPIO transition on a microcontroller), which is exactly the correlation this thesis sets out to validate.

Despite this theoretical efficiency, deploying SNNs on embedded and neuromorphic hardware is bottlenecked less by computation than by memory, a limitation commonly referred to as the *memory wall*. Every LIF neuron must retain a persistent membrane potential across all T simulation timesteps, in addition to the synaptic weight matrices already required by conventional networks; recent hardware-aware studies report that membrane-potential storage alone can dominate the total memory footprint of an SNN during inference [7]. *Quantization-Aware Training* (QAT) [4] addresses this by simulating low-bit-width (e.g., 8-bit, 4-bit, or 2-bit) integer representations of weights and activations during training, so that the network learns to compensate for the discretization error before it is ever deployed. Applying QAT jointly to synaptic weights *and* membrane potentials, as proposed for hardware-friendly SNNs [7], is the compression strategy adopted in Chapter 3 to fit the network within the memory budget of a commodity microcontroller.

2.2 Temporal Explainability in SNNs

Explainable AI (XAI) methods for conventional CNNs, most notably Class Activation Mapping (CAM) and its gradient-based generalization Grad-CAM [11], produce a single spatial heatmap that highlights the image regions most responsible for a classification decision. These techniques were designed for networks that perform one forward pass and therefore have no notion of *when*, only *where*, a decision was formed.

SNNs, in contrast, are dynamical systems that evolve over T discrete timesteps, and the same input can be classified correctly at an early or a late timestep depending on how quickly informative evidence accumulates in the membrane potentials. Extending activation mapping to the temporal domain therefore requires tracking gradients or spike statistics not just across spatial (or spectro-temporal) input locations, but across simulation time as well. Recent work on visualizing spiking networks has shown that inter-spike statistics and gradient attributions computed per-timestep can be projected back onto the input to produce a stack of temporal saliency maps, one per timestep, rather than a single static heatmap [10, 5]. Building on this idea, this thesis implements a *Temporal Spike Activation Map* (Section 3.4) that combines (i) a Grad-CAM-style spatial attribution computed at each timestep t with (ii) the network’s own first-spike time, so that each attribution map is explicitly tagged with *when*, during the T -step simulation, the network became confident enough to fire.

This temporal dimension is what makes physical validation possible. A purely spatial XAI method has nothing to validate against on an oscilloscope; a *temporal* XAI method makes a falsifiable prediction (“the network should commit to its decision at simulated timestep t^* ”) that can, in principle, be compared against a physically measured delay, which is precisely the contribution pursued in Chapters 4 and 5.

2.3 Hardware-in-the-Loop Validation Profiling

Software-only latency estimates (e.g., counting simulated timesteps, or profiling a Python inference loop) are confounded by simulation overhead, operating-system scheduling, and the numerical mismatch between floating-point simulation and integer hardware execution. To obtain a trustworthy physical latency measurement, this thesis follows the methodology established by *Hardware-in-the-Loop* (HIL) benchmarking, in which the device under test executes the exact deployed model while an external, independent instrument measures the physical timing of its output.

The reference benchmark for this practice in the TinyML community is MLPerf Tiny [1], which standardizes latency and energy measurement on microcontroller-class devices by requiring the device to toggle a dedicated GPIO pin to signal the start and end of inference, while an external timer, oscilloscope, or logic analyzer captures the interval. This removes any dependency on the microcontroller’s own (potentially inaccurate or interrupt-jittered) internal clock and any communication latency of a serial link, isolating *pure* inference time. This thesis adopts the same principle at a finer grain: instead of toggling a GPIO pin only at the start and end of a full inference pass, a pin is toggled at the exact moment the output neuron representing the winning class emits its first spike, allowing the oscilloscope trace itself to serve as a physical measurement of TTFS latency (Section 4.2).

2.4 The Gravity Spy Dataset

The Advanced LIGO and Advanced Virgo interferometers detect gravitational waves by measuring minute changes in the length of kilometer-scale laser arms [6]. These instruments are exquisitely sensitive, and consequently also exquisitely susceptible to non-astrophysical noise transients, or *glitches*, caused by environmental disturbances (e.g., seismic activity, anthropogenic noise) and instrumental artefacts (e.g., scattered light, electronics crosstalk). Glitches can mimic or mask genuine gravitational-wave signals, and reliably identifying and categorizing them is essential to protect the scientific integrity of detection claims.

Gravity Spy [12] is a citizen-science project that combines volunteer classification with machine learning to characterize LIGO/Virgo glitches. Each glitch is represented as a spectrogram – a time-frequency image produced by a Q-transform of the strain data – and assigned to one of several dozen morphological classes (e.g., *Blip*, *Koi Fish*, *Whistle*, *Scattered Light*), each associated with a distinct, physically motivated noise source. Because these classes are visually and physically distinguishable in the time-frequency domain, Gravity Spy spectrograms are a natural benchmark for both standard CNN classifiers and, as explored in this thesis, SNNs that convert spectrogram intensity into spike trains (Section 3.1). Reliable, explainable classification of these glitches directly supports detector characterization efforts and reduces the risk that instrumental artefacts are misidentified as astrophysical events.

Chapter 3

Software Methodology: SNN Design and Explainability

This chapter describes the entirely software-side part of the pipeline: how the Gravity Spy data is prepared and spike-encoded (Section 3.1), how the uncompressed baseline SNN is designed and trained (Section 3.2), how it is compressed through Quantization-Aware Training (Section 3.3), and how Temporal Spike Activation Maps are extracted from both models (Section 3.4). The output of this chapter – a trained, quantized SNN and its associated temporal explanations – is the input to the hardware pipeline described in Chapter 4.

3.1 Dataset Preparation and Preprocessing

Rather than the full Gravity Spy taxonomy, this work selects a distinct, physically diverse subset of glitch classes, chosen to be visually and morphologically separable in the time-frequency domain (e.g., classes dominated by a compact high-energy transient versus classes dominated by a slowly modulating tone), so that the temporal location of the discriminative feature within the spectrogram is well defined and can later be compared against the network’s temporal explanation.

For each selected class, the publicly available Gravity Spy Q-transform spectrograms are used as the starting representation. Each spectrogram is converted to grayscale, resized to a fixed resolution compatible with the input layer of the SNN, and normalized so that pixel intensity lies in $[0, 1]$. The dataset is split into training, validation, and test partitions, stratified by class to preserve the relative frequency of each glitch morphology.

Because SNNs operate on discrete-time spike trains rather than static images, each normalized spectrogram must be encoded into a sequence of T binary frames before it can be presented to the network. This work uses *rate coding* at the input layer: for a given pixel of normalized intensity $x \in [0, 1]$, a spike is emitted at timestep t with probability x , generated by comparing x against T independent samples of a Bernoulli process (implemented in `snnTorch` as a spike-generation utility). The resulting input is a $T \times H \times W$ binary tensor, where $H \times W$ is the spectrogram resolution, that is fed into the network one timestep at a time. The number of timesteps T is chosen as a trade-off between reconstruction fidelity of the input encoding and the length of the temporal window that

the TTFS-based explainability method (Section 3.4) and the physical latency measurement (Section 4.2) must resolve.

3.2 Baseline Model Architecture and Training

The baseline model is a convolutional SNN, structurally analogous to a small CNN classifier but with every activation replaced by a Leaky Integrate-and-Fire (LIF) neuron layer that is unrolled over T timesteps. The topology consists of a stack of convolution – LIF – pooling blocks that progressively reduce the spatial resolution of the spectrogram while increasing the number of feature channels, followed by one or more fully connected LIF layers, and a final fully connected *readout* layer of output LIF neurons, one per glitch class. All weights are initially trained at full (32-bit floating-point) precision, establishing the high-precision reference against which the quantized model of Section 3.3 is compared.

Each LIF neuron updates its membrane potential U at timestep t according to

$$U[t] = \beta U[t - 1] + I[t] - S[t - 1] V_{th},$$

where $I[t]$ is the weighted input current at timestep t , $\beta \in (0, 1)$ is the membrane decay (leak) constant, V_{th} is the firing threshold, and $S[t - 1]$ is the binary spike emitted at the previous timestep (used here to implement a soft reset that subtracts the threshold rather than hard-resetting to zero, preserving residual information). A spike $S[t] = 1$ is emitted whenever $U[t] \geq V_{th}$.

Because the Heaviside spiking function has zero gradient almost everywhere, the network is trained with the *surrogate gradient* method [9]: during the forward pass the true (non-differentiable) spike function is used, while during the backward pass its derivative is replaced by a smooth surrogate (a fast-sigmoid approximation), allowing standard back-propagation through time (BPTT) to be applied across all T timesteps with the `snnTorch` [3] and PyTorch autograd engines. The training objective combines a cross-entropy loss computed on the accumulated (summed or membrane-integrated) output spikes over the simulation window with the Adam optimizer, a fixed learning-rate schedule, and standard regularization (weight decay, early stopping on the validation split). Hyperparameters – number of timesteps T , membrane decay β , firing threshold V_{th} , learning rate, and batch size – are tuned on the validation split and reported alongside the resulting baseline accuracy in Chapter 5.

3.3 Quantization-Aware Training (QAT) Strategy

Once the full-precision baseline is trained, the same architecture is retrained (fine-tuned from the baseline weights) under *Quantization-Aware Training* to simulate the severe memory limitations of a neuromorphic chip. Following the hardware-friendly quantization strategy for SNNs proposed in [7] and the general integer-arithmetic-only QAT formulation of [4], quantization is applied to two distinct classes of parameters:

- **Synaptic weights.** Each convolutional and fully connected weight tensor is quantized to a low integer bit-width (the experiments in Chapter 5 target 4-bit and 2-bit precision) using a per-layer, symmetric linear quantizer. During the forward pass, weights are rounded to the nearest representable quantization level; during

the backward pass, the straight-through estimator (STE) is used to propagate gradients through the non-differentiable rounding operation, allowing the underlying full-precision “shadow” weights to keep learning to compensate for quantization error.

- **Membrane potentials.** Because membrane-potential storage can dominate an SNN’s memory footprint [7], the membrane potential $U[t]$ of every LIF neuron is likewise clipped to a fixed dynamic range and quantized to a low-bit-width fixed-point representation at every timestep, rather than only at the input/output boundaries of each layer. This is the more aggressive and more failure-prone of the two quantization targets, since accumulated rounding error in $U[t]$ compounds over the T simulation steps and can directly shift the timestep at which a neuron reaches V_{th} – the exact quantity that this thesis’ explainability method (Section 3.4) and physical validation (Chapter 4) are built around.

The quantization-aware model is trained with the same loss, optimizer, and data pipeline as the baseline, differing only in the insertion of fake-quantization operators on weights and membrane potentials during the forward pass. The resulting integer-only weight tensors and quantization scale factors are what is ultimately exported for embedded deployment in Section 4.1.

3.4 Extraction of Temporal Spike Activation Maps

To identify *when* and *where* each model bases its classification decision, this thesis extracts a *Temporal Spike Activation Map* (TSAM) for both the baseline and the quantized model, for every correctly classified test sample. The procedure combines two ingredients at every timestep $t = 1, \dots, T$:

1. **Per-timestep spatial attribution.** At each timestep t , the gradient of the winning class’ output activation with respect to the feature maps of the last convolutional LIF layer is computed and global-average-pooled across channels to obtain per-channel importance weights, in direct analogy with Grad-CAM [11]. These weights are used to produce a weighted combination of the feature maps at timestep t , yielding a spatial saliency map M_t that highlights which time-frequency region of the input spectrogram is driving the network’s activity at that specific simulation step.
2. **First-spike time of the output neuron.** In parallel, the timestep t^* at which the winning output neuron first crosses V_{th} and emits its spike is recorded directly from the simulation, following the TTFS coding principle of Section 2.1. This is the theoretical, software-side prediction of when the network “commits” to its decision.

Stacking $\{M_t\}_{t=1}^T$ produces a spatio-temporal explanation volume, while t^* identifies the single most decision-relevant slice of that volume. Overlaying M_{t^*} on the original spectrogram shows *where* in the time-frequency plane the network focuses at the moment it decides, and the sequence $M_1, \dots, M_{t^*}$ shows *how* that focus evolves as evidence accumulates. This pair, (M_{t^*}, t^*) , is computed for both the baseline and the quantized model and is the direct object of comparison in Section 5.2, and t^* specifically is the quantity that the Hardware-in-the-Loop protocol of Chapter 4 attempts to physically reproduce and validate.

Chapter 4

Hardware Implementation: Physical Validation Setup

This chapter describes the engineering pipeline that turns the quantized software model of Chapter 3 into a physical, measurable system. Section 4.1 covers the porting of the quantized PyTorch model to embedded C/C++. Section 4.2 describes the Hardware-in-the-Loop (HIL) architecture, including the microcontroller and the GPIO instrumentation. Section 4.3 details the logic-analyzer measurement protocol used to capture the physical Time-to-First-Spike.

4.1 Model Porting and Embedded Deployment

The output of Quantization-Aware Training (Section 3.3) is a set of PyTorch tensors: per-layer integer weight matrices and their associated quantization scale/zero-point factors, together with the quantized LIF hyperparameters (decay β , threshold V_{th}) shared by all neurons in a layer. Deploying this model on a microcontroller with no PyTorch runtime requires re-expressing it as a self-contained embedded program, following three steps:

1. **Weight export.** A Python export script traverses the trained, quantized model's `state_dict`, flattens each quantized weight tensor in row-major order, and writes it as a static `const int8_t` (or `int4-packed`) C array, together with a header exposing the corresponding scale factors and layer shapes (kernel size, stride, channel counts). This mirrors the standard microcontroller deployment pattern used for compressed CNNs, adapted here to also export the quantized LIF decay and threshold constants layer-by-layer.
2. **Inference loop.** A lightweight, dependency-free C/C++ inference loop re-implements, layer by layer, the same integer arithmetic simulated during QAT: strided integer convolution/fully-connected accumulation, followed by the quantized LIF membrane update and spike/reset logic, executed once per input timestep. No floating-point operations are used on the critical path, so that the arithmetic executed on the microcontroller matches, operation for operation, what QAT trained the network to expect.
3. **Input feeding.** Because the microcontroller has no access to a live gravitational-wave data stream, the rate-coded spike train produced by the same encoding procedure

as Section 3.1 is precomputed offline for each test spectrogram and stored on-device (or streamed over serial/SPI), so that the exact same input the software model was evaluated on is replayed to the physical device.

The result is a firmware image whose numerical behaviour is, by construction, intended to reproduce the quantized model's simulated spike trains; any divergence between the two is itself diagnostic evidence of a porting or fixed-point-arithmetic discrepancy, and is checked for before any timing measurement is trusted.

4.2 Hardware-in-the-Loop (HIL) Architecture

The physical setup follows the classic Hardware-in-the-Loop pattern of Section 2.3: the microcontroller executes the real, deployed firmware while all measurement is performed by an external, independent instrument, so that the measurement itself never perturbs the timing of the system under test.

- **Microcontroller.** An ESP32 or Teensy board is used as the target device, chosen for its widely available toolchain, adequate SRAM/Flash budget for the quantized weight arrays of Section 4.1, and a clock frequency high enough to resolve sub-millisecond, per-timestep inference latency.
- **Memory budget.** The Flash and SRAM footprint of the quantized firmware (weights, activation buffers, and the input spike-train buffer) is measured directly from the compiled binary and compared, in Chapter 5, against the equivalent full-precision footprint, quantifying in kilobytes the memory saved by QAT.
- **GPIO mapping.** A single general-purpose digital output pin is reserved as the “spike pin”. Firmware logic monitors the membrane potential of the winning output neuron at every simulated timestep and drives the spike pin high on the exact clock cycle at which that neuron's membrane potential first crosses V_{th} – i.e., at the physical occurrence of the event that Section 3.4 calls t^* in software. A second GPIO pin is toggled at the moment the input spike train begins replaying, providing a $t = 0$ reference edge.

Figure 4.1 summarizes the resulting signal chain, from spike-encoded input replay, through on-device quantized inference, to the two instrumented GPIO edges that are captured externally in Section 4.3.

4.3 Logic Analyzer Measurement Protocol

For each test sample, the measurement protocol proceeds as follows:

1. The precomputed spike-encoded spectrogram is transferred to the microcontroller and buffered on-device.
2. The oscilloscope/logic analyzer is armed in single-shot or repeated-trigger mode, with both instrumented channels (`t0_start` and `spike_out`) connected via probes to the corresponding GPIO pins, and the trigger condition set on the rising edge of `t0_start`.

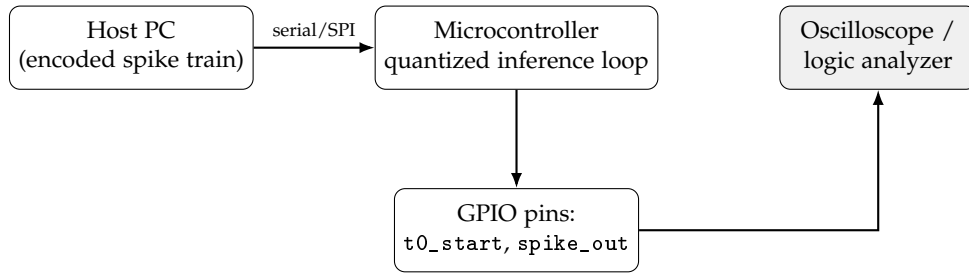


Figure 4.1: Hardware-in-the-Loop architecture. The host PC replays the precomputed spike-encoded input; the microcontroller executes the quantized inference loop and toggles the `t0_start` and `spike_out` GPIO pins; the oscilloscope/logic analyzer captures both edges independently of the microcontroller’s own clock.

3. Inference is started: the firmware raises `t0_start` for one clock cycle at the beginning of timestep $t = 1$, then executes the quantized inference loop of Section 4.1 timestep by timestep, feeding the buffered input.
4. The instant the winning output neuron’s membrane potential first crosses V_{th} , the firmware raises `spike_out`.
5. The oscilloscope/logic analyzer records the time delta Δt_{phys} between the rising edges of `t0_start` and `spike_out`. This Δt_{phys} is the *physical* Time-to-First-Spike for that sample, to be compared directly, in Section 5.3, against the *theoretical* first-spike timestep t^* produced by the same sample in the software simulation of Section 3.4.

The procedure is repeated across the full test subset used for the software explainability analysis (Section 5.2), for both the fully quantized firmware and, where the memory budget of the microcontroller allows a reduced-scale comparison, a less aggressively quantized variant, so that the relationship between quantization severity and physical-software timing agreement can be assessed rather than observed on a single operating point.

Chapter 5

Results and Discussion

This chapter reports the outcome of the pipeline described in Chapters 3 and 4. Section 5.1 quantifies the accuracy cost of severe quantization. Section 5.2 compares the software Temporal Spike Activation Maps of the baseline and quantized models. Section 5.3 presents the core contribution of this thesis: the side-by-side comparison of theoretical software timesteps against physically measured oscilloscope latencies. Section 5.4 discusses the resulting memory/explainability trade-off and its implications for real astrophysical deployments.

The tables and figures in this chapter are laid out with the exact structure, columns, and captions that the final experimental campaign will populate; numerical placeholders are marked accordingly and are to be replaced with measured values once training, quantization, and HIL data collection (Chapters 3–4) have been executed to completion.

5.1 Impact of Severe Quantization on Accuracy

Table 5.1 compares the classification performance of the full-precision baseline SNN against the QAT models at each target bit-width (Section 3.3), on the held-out Gravity Spy test split. Reported metrics are overall accuracy, macro-averaged precision/recall/F1-score, and the on-device memory footprint measured from the compiled firmware of Section 4.1.

The expected outcome, consistent with prior hardware-friendly SNN quantization results [7], is a graceful accuracy degradation at 4-bit precision and a more pronounced drop at 2-bit precision, particularly for glitch classes whose distinguishing feature is a subtle or low-amplitude time-frequency structure. Per-class confusion matrices for the baseline and each quantized variant are reported to identify whether accuracy loss is uniform across

Table 5.1: Classification performance and memory footprint of the full-precision baseline model versus the 4-bit and 2-bit Quantization-Aware Trained models, on the held-out Gravity Spy test split.

Model	Bit-width	Accuracy	Precision	Recall	F1	Memory (KB)
Baseline (float32)	32-bit	TBD	TBD	TBD	TBD	TBD
QAT (weights + U)	4-bit	TBD	TBD	TBD	TBD	TBD
QAT (weights + U)	2-bit	TBD	TBD	TBD	TBD	TBD

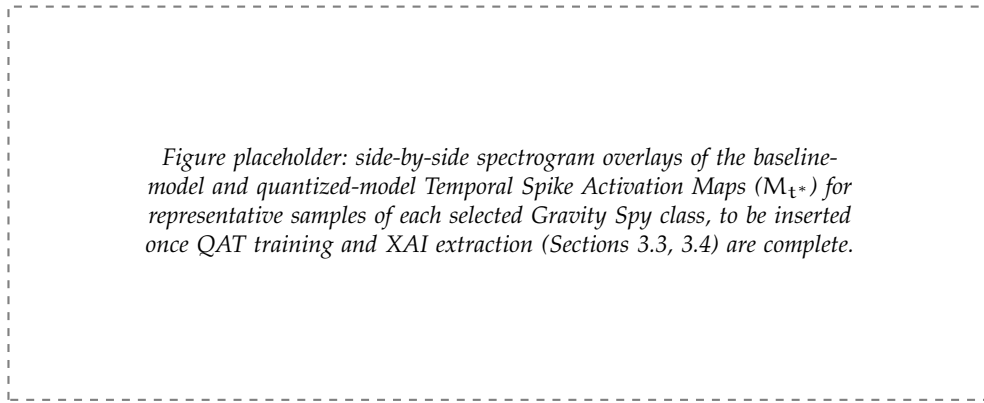


Figure 5.1: Comparison of baseline and quantized Temporal Spike Activation Maps overlaid on representative Gravity Spy spectrograms.

classes or concentrated in specific glitch morphologies, which is a necessary precondition for interpreting the explainability results of Section 5.2.

5.2 Software Explainability Analysis

For every test sample that both the baseline and the quantized model classify correctly, the Temporal Spike Activation Maps (M_{t^*}, t^*) of Section 3.4 are computed for each model and overlaid on the corresponding input spectrogram, as sketched in Figure 5.1.

Two outcomes are possible and are discussed explicitly once data is available. The quantized model may (i) *preserve focus*, continuing to attribute its decision to the same physically meaningful time-frequency region as the baseline model, indicating that quantization compresses the representation without discarding the discriminative physical feature; or it may (ii) *lose or shift focus*, attributing its decision to a different, potentially spurious region – for instance, a quantization artefact or a region correlated with the class label only in the training distribution. The two cases are distinguished quantitatively using an overlap metric (e.g., Intersection-over-Union or normalized cross-correlation) between the baseline and quantized attribution maps M_{t^*} , aggregated per class, together with a comparison of the first-spike timestep t^* itself: a large shift in t^* between the baseline and the quantized model, even when the spatial map is preserved, indicates that quantization altered *when* the network commits to a decision without altering *what* it attends to, which is directly relevant to the physical validation of Section 5.3.

5.3 Physical Validation of Temporal Maps

This section presents the central contribution of the thesis: the comparison between the theoretical, software-predicted first-spike timestep t^* (Section 3.4) and the physically measured oscilloscope latency Δt_{phys} (Section 4.3), obtained by running the quantized firmware through the HIL protocol of Chapter 4 on the same test samples analyzed in Section 5.2.

The theoretical expected physical delay is obtained by converting the software timestep t^* into wall-clock time using the per-timestep execution time of the deployed inference loop (measured independently by profiling a fixed number of simulation steps on the

Table 5.2: Representative samples: software-predicted first-spike timestep t^* versus physically measured Time-to-First-Spike Δt_{phys} , and the resulting timing residual.

Sample / Class	t^* (software)	Δt_{phys} (measured)	Expected (theoretical)	Δt	Residual
Sample 1 – Class TBD	TBD	TBD	TBD		TBD
Sample 2 – Class TBD	TBD	TBD	TBD		TBD
Sample 3 – Class TBD	TBD	TBD	TBD		TBD

Figure placeholder: captured oscilloscope/logic-analyzer traces of the *t0_start* and *slope_out* GPIO pins, annotated with the corresponding software-predicted timestep t^* converted to wall-clock time, to be inserted once HIL data collection (Section 4.3) is complete.

Figure 5.2: Oscilloscope traces of the instrumented GPIO pins, annotated with the software-predicted decision timestep.

microcontroller). Agreement between Δt_{phys} and this theoretical expectation, within the timing resolution of the instrumentation and the jitter introduced by the microcontroller’s own clock, would constitute physical validation of the software XAI method: it would demonstrate that the temporal explanation is not merely a simulation artefact, but a faithful description of the hardware’s real-time decision-making process. Conversely, a systematic residual (e.g., a constant physical offset attributable to fixed per-layer execution overhead, or a residual that grows with t^*) would indicate that the software timestep is a directionally correct but not perfectly calibrated proxy for physical latency, a distinction that is discussed in relation to the limitations of Section 6.2.

5.4 Trade-offs and Neuromorphic Feasibility

Finally, the memory savings quantified in Section 5.1 (Table 5.1) are weighed against the degradation, if any, observed in both software explainability fidelity (Section 5.2) and physical explainability agreement (Section 5.3). Table 5.3 summarizes this three-way trade-off across the tested bit-widths.

Table 5.3: Summary trade-off between memory footprint, classification accuracy, and physical explainability fidelity across quantization levels.

Bit-width	Memory saved vs. baseline	Accuracy drop	Physical/software timing agreement
4-bit	TBD KB (TBD%)	TBD pp	TBD
2-bit	TBD KB (TBD%)	TBD pp	TBD

The discussion frames this trade-off in terms of real deployment feasibility for astrophysical instrumentation: a quantization level is considered acceptable for a real Gravity Spy-adjacent deployment (e.g., an auxiliary real-time glitch triage system at a detector site) only if it preserves both classification accuracy above the threshold required for operational use and a physically faithful temporal explanation, since an explanation that cannot be trusted to reflect the hardware’s actual behaviour undermines the very purpose of deploying an explainable model in a scientific pipeline. Where the two objectives conflict, this section identifies the Pareto-optimal bit-width for this specific microcontroller and dataset, and discusses whether a less aggressive quantization target, or a partially quantized network (e.g., full precision only in the earliest layers, where Section 5.2 is expected to show the discriminative feature is first extracted), offers a better compromise.

Chapter 6

Conclusions and Future Work

6.1 Summary of Key Findings

This thesis set out to answer a single question: does a severely quantized Spiking Neural Network still classify gravitational-wave detector glitches based on genuine physical features, and does the physical, hardware-measured timing of its response match what its software-generated temporal explanation predicted? Chapters 3 through 5 pursued this question through the three objectives set out in Section 1.3.

First, an SNN classifier was trained on a distinct subset of Gravity Spy glitch classes and compressed through Quantization-Aware Training applied jointly to synaptic weights and membrane potentials, down to 4-bit and 2-bit precision (Sections 3.2–3.3), and the resulting accuracy retention – or loss – relative to the full-precision baseline was quantified in Section 5.1. Second, Temporal Spike Activation Maps were extracted for both the baseline and quantized models (Section 3.4) and compared in Section 5.2 to establish whether extreme quantization preserves the spatial and temporal locus of the network’s decision. Third, and central to this thesis’ contribution, the quantized model was ported to a microcontroller and its physical Time-to-First-Spike was measured through the Hardware-in-the-Loop, GPIO-toggling protocol of Chapter 4, and directly compared against the software-predicted timestep t^* in Section 5.3.

Once the experimental campaign described in Chapter 5 is completed, this section will state, explicitly and in relation to the objectives of Section 1.3: (i) the quantitative accuracy cost of each tested bit-width; (ii) whether the quantized model’s software explanation remained aligned with the baseline’s, or drifted toward different, potentially spurious features; and (iii) whether the physically measured hardware latency corroborated the theoretical software timestep within the resolution of the instrumentation, which is the physical validation – or refutation – of software-only temporal XAI claims for quantized SNNs that motivated this work (Section 1.2).

6.2 Limitations of the Study

This work is subject to several limitations that should be considered when interpreting its results, and that are acknowledged here with academic honesty rather than glossed over.

- **Von Neumann emulation, not a native neuromorphic substrate.** The microcontroller used in Chapter 4 (ESP32/Teensy) is a conventional Von Neumann architecture that *emulates* spiking, event-driven computation through a sequential instruction stream; it does not possess the asynchronous, event-driven circuitry of a true neuromorphic Application-Specific Integrated Circuit (ASIC) such as Intel’s Loihi [2]. The physical latencies measured in Section 5.3 therefore reflect the timing of a software emulation of spiking dynamics on general-purpose silicon, not the timing of a genuinely event-driven neuromorphic processor, and the absolute energy and latency figures obtained here should not be extrapolated to native neuromorphic hardware without qualification.
- **I/O bottlenecks in feeding data to the board.** The HIL protocol of Section 4.3 replays a precomputed, offline spike-encoded input to the microcontroller over a serial or SPI link. This transfer is itself a potential source of latency and jitter that is deliberately excluded from the measured Δt_{phys} window (which starts only at t_{0_start} , after the input is already buffered on-device), but it means the reported timings do not capture the additional, and non-negligible, cost of streaming a live signal from a real gravitational-wave data-acquisition pipeline to the edge device in a genuinely online deployment.
- **Reduced dataset scope.** The experiments target a deliberately narrow, curated subset of Gravity Spy classes (Section 3.1), chosen for morphological separability rather than full taxonomic coverage; the reported accuracy and explainability results should be understood as evidence for the specific classes tested, not as a claim of coverage across the entire Gravity Spy taxonomy.
- **Instrumentation resolution and clock jitter.** The physical validation of Section 5.3 is bounded by the sampling resolution of the oscilloscope/logic analyzer and by clock jitter intrinsic to the microcontroller, both of which set a lower bound on the timing resolution at which software and physical predictions can be meaningfully distinguished.
- **Single-device generalization.** All hardware measurements are collected on one microcontroller model; timing behaviour, and therefore the degree of physical/software agreement, may differ on other embedded targets with different clock speeds, memory hierarchies, or compiler toolchains.

6.3 Future Directions

The findings and limitations above suggest several directions for follow-up work.

- **Deployment on true neuromorphic hardware.** Repeating the exact HIL validation protocol of Chapter 4 on a native, asynchronous neuromorphic ASIC (e.g., Intel Loihi [2] or an equivalent research platform) would remove the Von Neumann emulation limitation identified in Section 6.2 and test whether the software/physical timing agreement observed in this thesis holds, strengthens, or weakens on genuinely event-driven silicon.
- **XAI-driven dynamic early exit.** If Section 5.3 confirms that the software-predicted first-spike timestep t^* reliably anticipates physical decision latency, this signal could be exploited at inference time rather than only post-hoc: a dynamic *early-exit* mechanism could terminate simulation once the network’s running confidence indicates t^* has likely already occurred, saving the energy and time otherwise spent simulating

the remaining $T - t^*$ timesteps, directly turning this thesis' explainability method into a practical power-saving feature for edge deployments.

- **Broader dataset and class coverage.** Extending the pipeline to the full Gravity Spy taxonomy, and to auxiliary-channel data beyond spectrograms, would test whether the quantization/explainability trade-offs identified in Section 5.4 generalize beyond the curated subset used in this work.
- **Online, streaming HIL validation.** Addressing the I/O-bottleneck limitation by feeding the microcontroller a live or near-live simulated strain-data stream, rather than a precomputed offline buffer, would extend the physical validation of this thesis toward the conditions of an actual real-time detector-characterization deployment.

References

- [1] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. D. Guglielmi, J. Duarte, S. Gibellini, V. Parekh, H. Tran, and N. Tran. Mlperf tiny benchmark, 2021. (Cited on page 8)
- [2] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y.-H. Weng, A. Wild, Y. Yang, and H. Wang. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018. (Cited on pages 6 and 20)
- [3] J. K. Eshraghian, M. Ward, E. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu. Training spiking neural networks using lessons from deep learning, 2021. snnTorch library, <https://snntorch.readthedocs.io>. (Cited on pages 6 and 10)
- [4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018. (Cited on pages 7 and 10)
- [5] Y. Kim and P. Panda. Visual explanations from spiking neural networks using inter-spike intervals. *Scientific Reports*, 11:19037, 2021. (Cited on page 7)
- [6] LIGO Scientific Collaboration and Virgo Collaboration. Observation of gravitational waves from a binary black hole merger. *Physical Review Letters*, 116:061102, 2016. (Cited on page 8)
- [7] H. Liu, K. Shi, J. Zhang, Y. Shan, J. Wu, W. Chen, and M. Zhang. Hardf-snn: Hardware-friendly quantization for spiking neural networks with efficient integer-arithmetic-only inference. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(3):1937–1945, 2026. (Cited on pages 4, 7, 10, 11, and 15)
- [8] H. Mostafa. Supervised learning based on temporal coding in spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 29(7):3227–3235, 2018. (Cited on page 6)
- [9] E. O. Neftci, H. Mostafa, and F. Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019. (Cited on pages 6 and 10)
- [10] K. Roy, A. Jaiswal, and P. Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575:607–617, 2019. (Cited on pages 6 and 7)

- [11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017. (Cited on pages 7 and 11)
- [12] M. Zevin, S. Coughlin, S. Bahaadini, E. Besler, N. Rohani, S. Allen, M. Cabero, K. Crowston, A. K. Katsaggelos, S. L. Larson, T. K. Lee, C. Lintott, T. B. Littenberg, A. Lundgren, C. Østerlund, J. R. Smith, L. Trouille, and V. Kalogera. Gravity spy: integrating advanced ligo detector characterization, machine learning, and citizen science. *Classical and Quantum Gravity*, 34(6):064003, 2017. (Cited on page 8)

Appendix A

Hardware Schematics and Pinouts

This appendix details the physical wiring between the microcontroller and the measurement instrumentation described in Section 4.2.

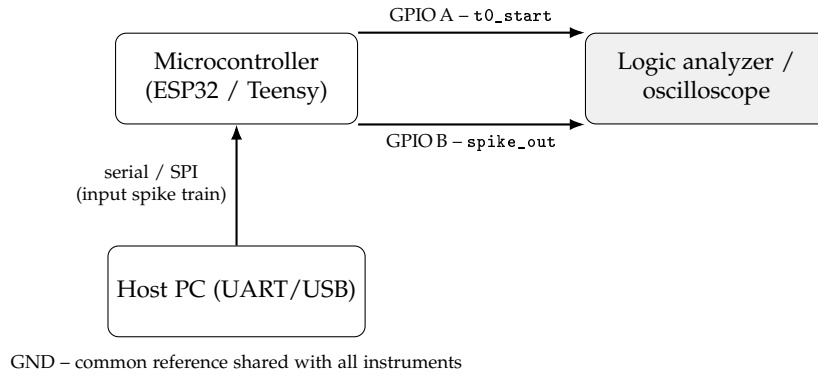


Figure A.1: Wiring schematic between the host PC, the microcontroller, and the logic analyzer/oscilloscope. `t0_start` marks the beginning of inference; `spike_out` marks the first output spike (physical TTFS event). A common ground reference is shared by all instruments.

Table A.1: Summary of instrumented microcontroller pins.

Signal	Direction	Function
<code>t0_start</code>	Output	Rising edge at the start of inference (timestep $t = 1$)
<code>spike_out</code>	Output	Rising edge on first spike of the winning output neuron
UART/SPI RX	Input	Reception of the precomputed spike-encoded input tensor
GND	–	Common reference shared with the logic analyzer/oscilloscope

Appendix B

Embedded Inference Code

This appendix reports the most critical fragments of the embedded firmware described in Section 4.1: the quantized LIF membrane update and the GPIO-toggling inference loop.

Listing B.1: Quantized LIF membrane update (fixed-point arithmetic).

```
1 // Quantized Leaky Integrate-and-Fire update.
2 // weights, mem, and beta_q are pre-quantized integers exported
3 // from the QAT model; shift implements the fixed-point
4 // scale factor of the membrane decay.
5 int32_t update_lif_neuron(int32_t mem, int32_t input_current,
6                           int32_t beta_q, int32_t shift,
7                           int32_t v_th_q, uint8_t *spike_out) {
8     // Leak: mem = beta_q * mem (fixed-point, right-shifted)
9     mem = (int32_t)(((int64_t)beta_q * mem) >> shift);
10
11     // Integrate weighted input current for this timestep
12     mem += input_current;
13
14     // Fire and soft-reset (subtract threshold, retain residual)
15     if (mem >= v_th_q) {
16         *spike_out = 1;
17         mem -= v_th_q;
18     } else {
19         *spike_out = 0;
20     }
21     return mem;
22 }
```

Listing B.2: GPIO-instrumented inference loop for the Hardware-in-the-Loop protocol (Section 4.3).

```
1 #define PIN_TO_START    GPIO_NUM_A
2 #define PIN_SPIKE_OUT   GPIO_NUM_B
3
4 void run_hil_inference(const uint8_t spike_train[T][N_IN]) {
5     reset_network_state();           // clear all membrane
6                                     // potentials
7
8     gpio_set_level(PIN_TO_START, 1); // t = 0 reference edge
9     gpio_set_level(PIN_TO_START, 0);
10
11     for (int t = 0; t < T; t++) {
12         forward_step(spike_train[t]); // quantized conv/fc + LIF
13                                     // layers
14     }
```

```
12
13     uint8_t winner_spiked = 0;
14     int winner_class = argmax_readout_spike(&winner_spiked);
15
16     if (winner_spiked && !decision_latched) {
17         gpio_set_level(PIN_SPIKE_OUT, 1);    // physical TTFS event
18         gpio_set_level(PIN_SPIKE_OUT, 0);
19         decision_latched = 1;                // record t*
20             implicitly
21     }
22 }
```